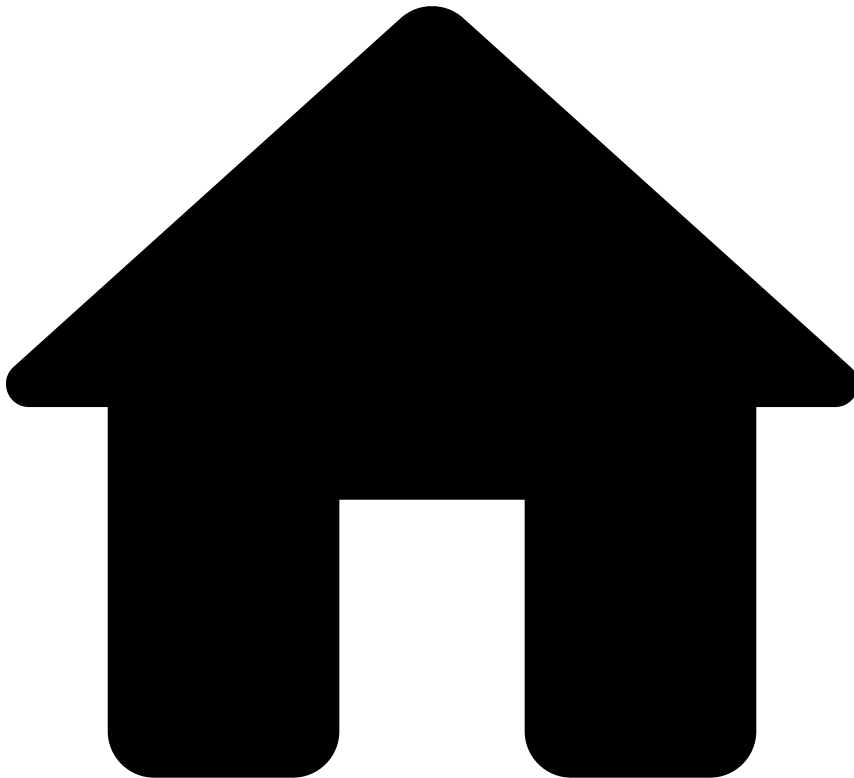


•



- [Rule Guide](#)
- Creating Rules

On this page

Was this page helpful? 

Creating a Rule

A rule, when used in a workflow, evaluates if a certain condition is true for the current transaction. According to the result, the rule can trigger actions in the workflow like rejecting or accepting a transaction, and contribute to the score of the transaction.

The following sections describe how to create a new rule, or edit an existing one after adding new features.

How to

To create a new rule from scratch, execute the following actions:

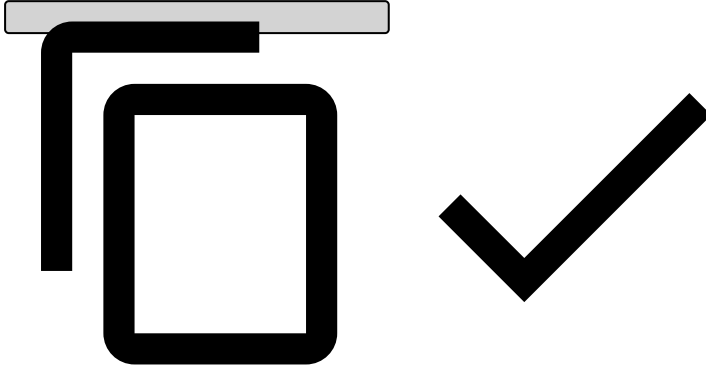
1. In the **Data Science** tab, select an existing rules project, and add a new rule.

2. Give your rule a **Name**, and describe it in a **Comment**.

3. Write the [Rule Definition Language \(RDL\) expression](#) for the condition of the rule. If a transaction matches the condition, it triggers the rule.

For example, to check if the current transaction's card number is on the watch list, the condition is:

```
CardWatchList.contains(TokenizedCardPAN)
```

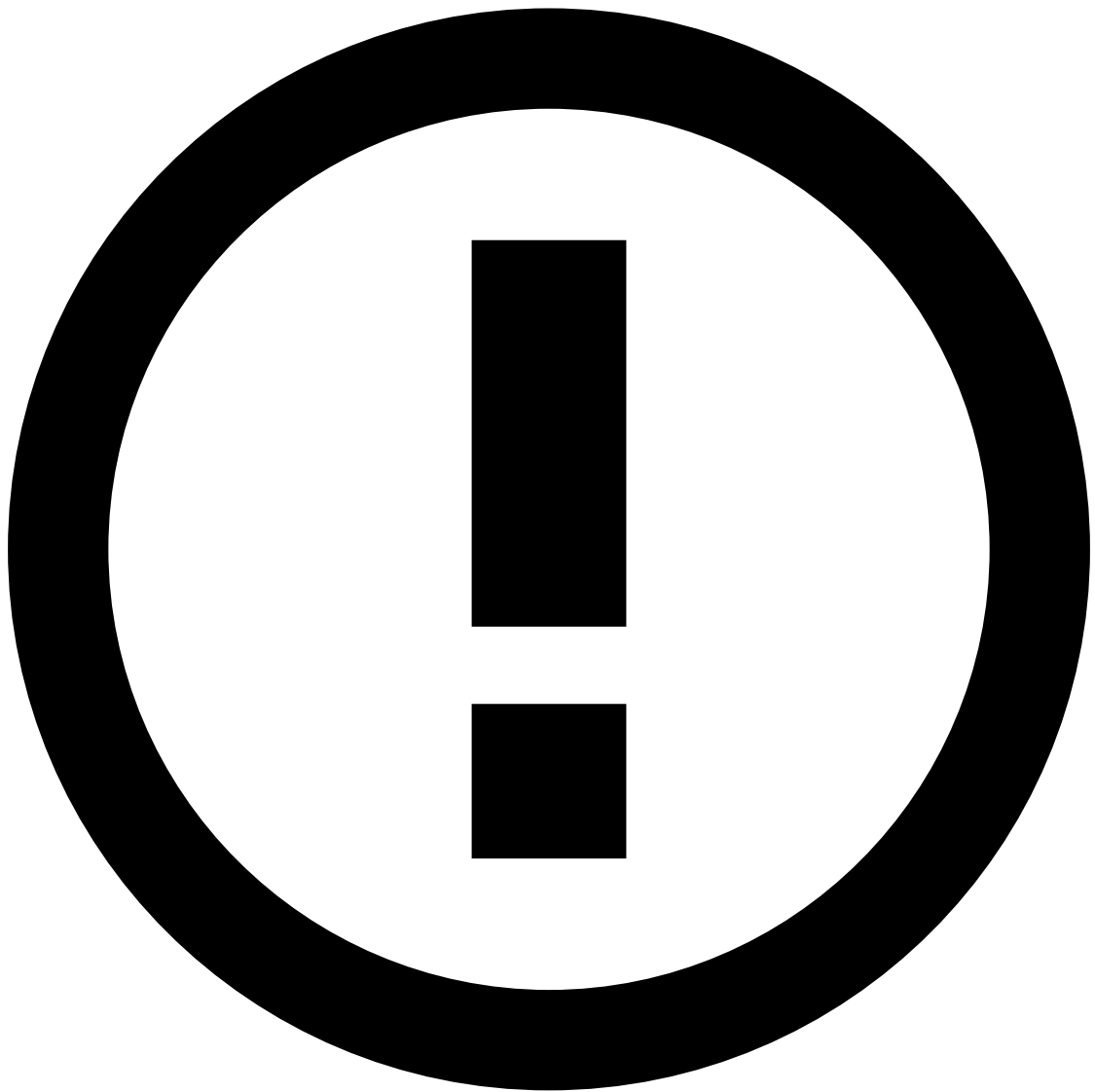




Best practice

Although it is possible to write complex expressions in the condition of a rule, it is recommended to [create a feature](#) instead, using a map or metrics operator in a plan.

You have access to **Lists** in the drop-down menu as shown in the figure below. If the type of your rules project is *Global*, you only have access to global lists. If the type of rules project is *Duplicated for Tenant* or *By Tenant*, you have access to all lists.



Rule Definition Language

Check the following pages to learn how to write expressions for rules:

- [Operators in rules](#)
- [Data types in rules](#)
- [Built-in UDEs](#)

4. Optionally, write an **Explanation**. The explanation can be shown to users who review transactions when the rule is triggered. You can type the explanation text and use data field values, list items, and variable values to give details to the reader.
5. The following four configuration options are available for you to control what consequences it has when the rule is triggered. They can be combined according to your business needs:
 1. Optionally, you can select which **Actions** of the rules project will the rule generate when triggered in a workflow.

Actions work like events, to which a custom workflow service can listen. For example, you can trigger the *Suspicious-transaction* action in the current rule. If you have a custom workflow service listening to the *Suspicious-transaction* action, the custom workflow service can send that transaction to an external

review system for analysis. A typical example of this is the Alert Storage Service, which sends the suspicious transactions to Case Manager.

2. Optionally, in the **Mark As** field you can select a value for the target variable field. This way you can mark an event as potentially fraudulent, or as a high risk transaction - depending on your use case and the target variable of your rules project.

If you don't select a value in the *Mark As* field, the rule won't give any value to the target variable.

Note that the possible values of the *Mark As* field come from the field that you set as target variable when [creating the rules project](#).

3. Optionally, you can set how the triggering of a rule generates a **Score Contribution** to a workflow outcome.

The **Score** of the current rule will be combined with other scoring elements of the same workflow: models and other blocks of rules. The **Weight** you assign to the current rule can be taken into consideration when calculating the final score.

4. In **Control Flow**, set how the workflow should proceed after triggering and processing the current rule. For example, you can choose to immediately stop the workflow when the current rule is triggered and processed.

This is especially useful when you want to ignore the evaluation of further rules, or even abort the rest of the workflow when a rule is triggered. Available options are as follows:

- **Continue**

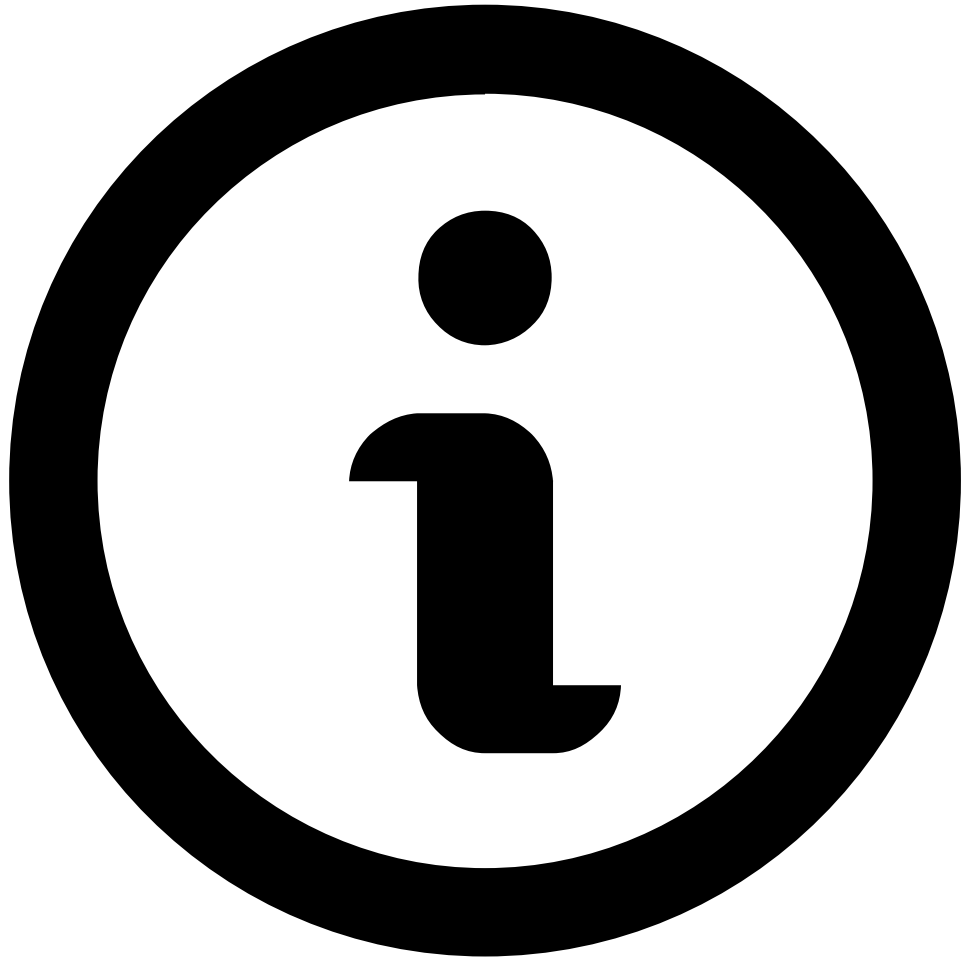
The execution continues with the next rule in the rule block. If there is no other rule in the rule block, then the workflow continues with the next workflow element.

- **Stop Workflow**

The workflow execution stops. Actions resulting from the current rule will be triggered for the current event.

- **Next Workflow Element**

No more rules are executed in this rule block. The execution continues with the next workflow element.



What happens to events after a workflow terminates?

The event is not lost. If the event storage is enabled in the workflow, once the event is out of the workflow it is stored in the database.

Editing a rule after adding new features🔗📄

If you added new features and want to include them in an existing rule, you just need to ensure that the origin schema of the rules project was updated. After that, the features will be available in the rules. You can edit the rule using the steps described in the previous section.

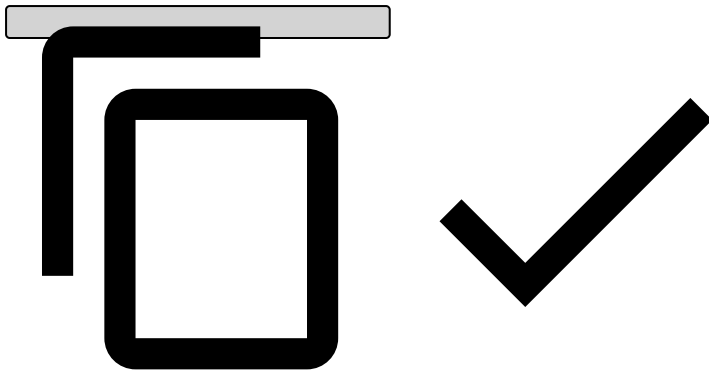
Examples of rules🔗📄

This section gives generic examples of conditions used in rules.

Count🔗📄

The following rule uses a [feature](#) called *Weekly_Count_per_Card_Pan*, which is the number of transactions in the last week with the same PAN card as the current transaction. The rule is triggered if the same card has been used more than twenty times in the last week.

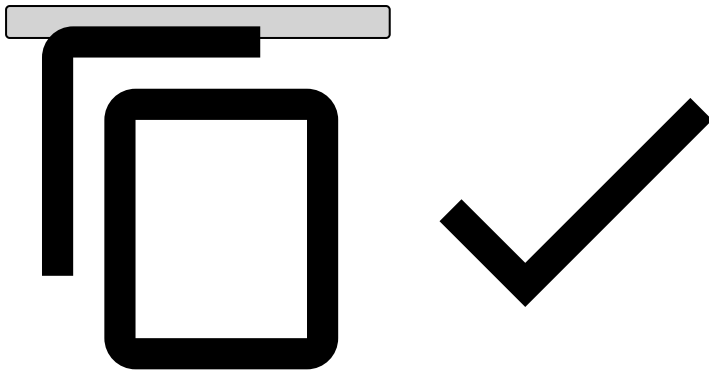
```
Weekly_Count_per_Card_Pan >= 20
```



Mismatch

The following rule uses two [features](#) that calculate the city of the customer's address, and of the device of the transaction. The rule triggers if the two cities don't match.

```
Customer_Addr_City != Device_IP_City
```

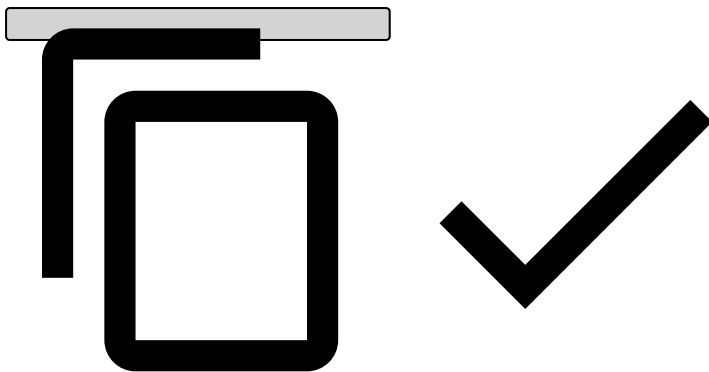


UDF call

To call a [User-Defined Function](#) (UDF) from RDL, the administrator has to define an alias for the UDF class in the Pulse properties.

The following rule calls a UDF that returns the keyboard distance of a string. The rule fires if the user's email address seems suspicious because the characters in it are very close to one another on a standard QWERTY keyboard.

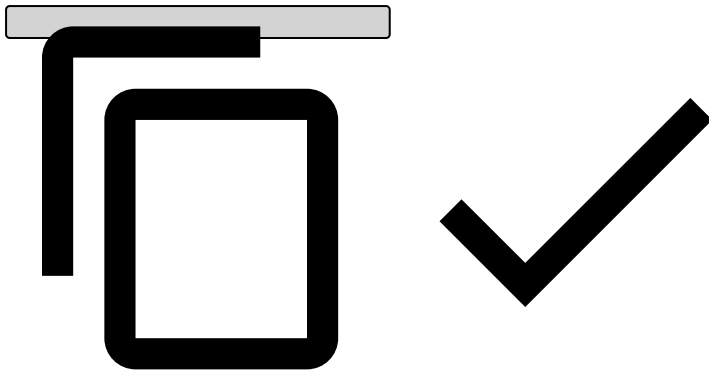
```
Strings.keyboardDistance(user_email_addr, 2) <= 5
```



List

The following rule condition uses a list. The rule fires if the IP address of the device associated to the current transaction is on a watch list.

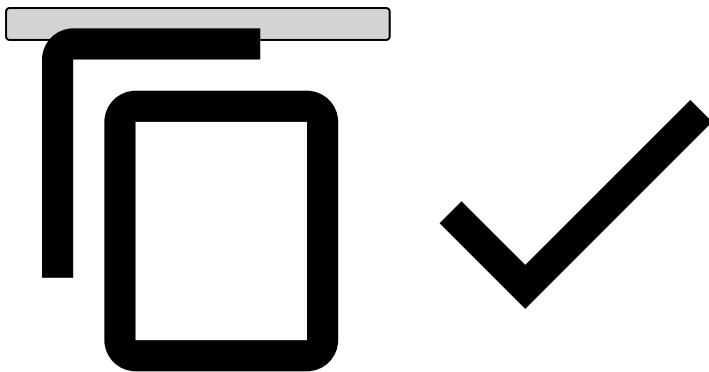
```
IP_WatchList.contains(Device_IP ?? "") // Tip: coalesce the argument to ensure it's not null.
```



Complex list

The following example shows the usage of a PosNeg list:

```
PosNeg_List.contains(Email_addr) and PosNeg_List[Email_addr].status == "block"
```



Using rule templates

Rule templates allow to create several rules with the same pattern, but with different values in custom variables. For example, you can define a rule template that checks if the transaction occurred in a certain currency, and the transaction amount is over a certain limit value. In this case, the currency and the limit value can be a parameter of the rule template. Then, you can instantiate this template with a different value for each country. This way you can have several instances of the same logic but with different parameters. Using a template in these cases avoids errors and allows you to maintain all the rule instances centrally in the template.

Example for rule templates

Let's see an example for checking if the transaction occurred in a certain currency, and the transaction amount is over a certain limit value.

The rule template, with two input parameters:

A rule that creates an instance of this template with a specific currency and a specific transaction amount: